Guia de Estudos: Exercícios de Swift (Básico a Intermediário) 🚀

Neste guia, você aprenderá conceitos essenciais de **Swift** e será desafiado a resolver exercícios para aplicar esses conhecimentos. Vamos abordar tópicos como **variáveis, condicionais, loops, funções, optionals, arrays e structs**.

---

1. Conceito: Variáveis e Constantes

Swift permite declarar **variáveis** e **constantes**. As variáveis podem ser alteradas ao longo do tempo, enquanto as constantes são valores imutáveis depois de atribuídos.

- **`var`**: Usado para declarar uma variável, cujo valor pode ser alterado.
- **`let`**: Usado para declarar uma constante, cujo valor não pode ser alterado após ser atribuído.

```
// Declarando uma constante e uma variável
let pessoa = "Alice"     // Constante
var ano = 2024         // Variável

// Alterando o valor da variável
idade = 2025
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/thebasics/

Exercício 1: Trabalhando com Variáveis e Constantes

Enunciado:
Crie uma constante `nome` e uma variável `idade`. Atribua valores a elas e imprima os valores na tela. Depois, altere o valor da variável `idade` e imprima novamente.
Objetivo: Trabalhar com **variáveis** e **constantes**.

---

2. Conceito: Condicionais (if/else)

As **condicionais** permitem executar um bloco de código com base em uma **condição**. O **if/else** é usado para tomar decisões, verificando se a condição é verdadeira ou falsa.

- **`if`**: Avalia uma condição.

- **`else`**: Define o que acontecerá quando a condição não for verdadeira.
- **`else if`**: Usado quando há mais de uma condição a ser verificada.

```
let numero = 5

if numero == 0 {
    print("O número é 0")
} else if numero == 1{
    print("O número é 1")
} else {
    print("O número não é 0 nem 1")
}
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/controlflow/

Exercício 2: Verificação de Par ou Ímpar

Enunciado:
Escreva um código que verifique se um número dado é **par** ou **ímpar**.
Objetivo: Praticar a estrutura de **condicionais** (`if/else`).

---

3. Conceito: Laços de Repetição (Loops)

Os **loops** permitem repetir um bloco de código várias vezes. O Swift oferece diferentes tipos de loops:

- **`for-in`**: Ideal para iterar sobre **coleções** ou **intervalos** de números.
- **`while`**: Executa enquanto uma condição for verdadeira.
- **`repeat-while`**: Executa pelo menos uma vez e depois verifica a condição.

```
for i in 1...10 {
    print(i)
}
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/controlflow/#Looping

---

Exercício 3: Imprimir Números de 1 a 10

Enunciado:
Crie um loop que imprima os números pares de **1 a 10** na tela.
Objetivo: Praticar o uso do loop **for-in**.

---

4. Conceito: Funções

As **funções** em Swift são blocos de código que podem ser chamados para **executar uma tarefa específica**. Elas podem receber **parâmetros** e retornar **valores**.

- `func`: Palavra-chave usada para declarar uma função.
- **Parâmetros**: Valores passados para a função.
- **Retorno**: A função pode retornar um valor com a palavra-chave `return`.

```
func adicionaUm(_ x: Int) -> Int {
    return x + 1
}
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/functions/

Exercício 4: Criando uma Função para Somar Números

Enunciado:
Crie uma função chamada `soma` que recebe dois números inteiros como parâmetros e retorna a soma deles.
Objetivo: Trabalhar com **funções** e **parâmetros**.

---

5. Conceito: Arrays

Um **array** é uma coleção ordenada de elementos do mesmo tipo. Os elementos podem ser acessados usando **índices**. Swift fornece várias operações para **adicionar, remover e acessar** elementos em um array.

- `[]`: Usado para definir um array.
- `append()`: Método para adicionar elementos a um array.
- **Índice**: Os índices começam em 0.

```
// Criando um array de nomes
let nomes = ["Alice", "Bob", "Carlos", "Diana", "Eduarda"]

// Acessando e imprimindo elementos do array
print(nomes[0])  // Imprime o primeiro nome: "Alice"
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/collectiontypes/

Exercício 5: Iterando sobre um Array de Nomes

Enunciado:
Crie um array contendo 5 nomes e use um **loop** para imprimir todos os nomes na tela.
Objetivo: Praticar o uso de **arrays** e **loops**.

---

6. Conceito: Dicionários

Um **dicionário** em Swift é uma coleção não ordenada de **pares chave-valor**. Ele é útil para armazenar dados que podem ser recuperados rapidamente por uma chave única.

- `[:]`: Usado para definir um dicionário.
- **Chave e valor**: Cada elemento do dicionário é um par chave-valor.

```
var paises = ["BR": "Brasil", "EUA": "Estados Unidos", "ARG":
"Argentina", "FRA": "França"]
print(paises["BR"]!) // Brasil
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/collectiontypes/

Exercício 6: Criando e Acessando um Dicionário

Enunciado:
Crie um dicionário que armazene nomes de **países** como chaves e suas **capitais** como

valores. Em seguida, imprima a capital de um país específico.
Objetivo: Trabalhar com **dicionários**.

---

7. Conceito: Optionals

Optionals são usados quando uma variável pode **não ter valor**. Uma variável optional pode ter `nil` ou conter um valor. Para acessar o valor de uma variável optional, você deve **desempacotá-la**.

- `?`: Indica que a variável é optional.
- `if let` **ou** `guard let`: Usados para **desempacotar** optionals com segurança.

```
// Criando uma variável optional
var nomeOpcional: String? = "Alice"

// Usando if let para desempacotar e acessar o valor
if let nomeDesempacotado = nomeOpcional {
    print("O nome é \(nomeDesempacotado)")
} else {
    print("Nome não definido")
}

// Usando guard let para desempacotar e acessar o valor
guard let nomeDesempacotado = nomeOpcional else {
    print("Nome não definido")
    return
}
print("O nome é \(nomeDesempacotado)")
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/optionals/

Exercício 7: Usando Optionals com `if let`

Enunciado:
Crie uma variável optional que armazene um nome. Se o nome não for **nil**, imprima na tela. Caso contrário, imprima uma mensagem dizendo que o nome não está definido. Tente trabalhar tanto com o if let como com o guard
Objetivo: Trabalhar com **optionals** e **desempacotamento seguro** (`if let`).

---

8. Conceito: Switch Case

O **switch case** permite testar várias condições de forma eficiente. Ele pode ser usado para verificar **intervalos**, **valores específicos**, ou **condições complexas**.

- **`case`**: Define um valor ou condição a ser verificada.
- **`default`**: Caso nenhum `case` seja verdadeiro.

```
let nome = "Mark"

switch nome {
case "Mark":
    print("Olá Mark!")
case "João":
    print("Olá João!")
default:
    print("Olá estranho!")
}
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/controlflow/#Switch

---

Exercício 8: Verificação de Nota com Switch

Enunciado:
Escreva um código que receba uma **nota** (A, B, C, D, F) e imprima uma mensagem correspondente ao conceito.
Objetivo: Trabalhar com a estrutura **switch case**.

---

9. Conceito: Tuplas

Uma **tupla** é um tipo de coleção que pode armazenar múltiplos valores de tipos diferentes. Diferente de arrays, tuplas podem conter valores de tipos variados, o que as torna mais flexíveis para certas situações.

- **`(valor1, valor2)`**: Sintaxe para criar uma tupla.

- **Acessando elementos**: Podemos acessar os elementos da tupla usando números de índice ou nomes.

```
// exemplo de tupla em swift
let http404Error = (404, "Not Found")
// exemplo de tupla em swift
print("The status code is \(http404Error.0)")
print("The status message is \(http404Error.1)")
// exemplo de tupla em swift
let (statusCode, statusMessage) = http404Error
print("The status code is \(statusCode)")
print("The status message is \(statusMessage)")
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/thebasics/#Tuples

---

Exercício 9: Usando Tuplas para Representar um Produto

Enunciado:
Crie uma tupla que represente um **produto**, contendo nome, preço e quantidade em estoque. Imprima os dados do produto.
Objetivo: Trabalhar com **tuplas**.

---

10. Conceito: Structs

Structs são tipos personalizados em Swift que permitem agrupar dados de diferentes tipos. Elas são muito úteis quando você deseja criar modelos de dados complexos.

- `struct`: Palavra-chave para definir uma struct.
- **Propriedades e métodos**: Uma struct pode conter **propriedades** e **métodos**.

```
struct Person {
    var name: String
    var age: Int
    var isStudent: Bool
}
let person = Person(name: "John", age: 25, isStudent: true)
print(person.name) // John
print(person.age) // 25
```

```
print(person.isStudent) // true
```

https://docs.swift.org/swift-book/documentation/the-swift-programming-language/classesandstructures/

Exercício 10: Criando uma Struct para um Carro

Enunciado:
Crie uma struct chamada `Carro` com propriedades `marca`, `modelo` e `ano`. Depois, instancie um carro e imprima seus dados.
Objetivo: Trabalhar com **structs** em Swift.

Desafio Swift: "Sistema de Cadastro de Livros" 📚

Objetivo: Criar um pequeno sistema para cadastrar e gerenciar informações de livros utilizando os conceitos aprendidos até agora em Swift. Você irá trabalhar com **variáveis**, **arrays**, **optionals**, **funções**, **dicionários**, **switch case**, **tuplas** e **structs**.

Enunciado:

Você precisa criar um sistema simples de cadastro de livros. O sistema deve permitir que o usuário adicione livros a uma lista, veja as informações dos livros cadastrados e permita que ele visualize o status de um livro (se está disponível ou emprestado).

Requisitos:

1. **Estrutura de Dados (struct)**:
   - Crie uma `struct` chamada `Livro` que tenha os seguintes atributos:
     - `titulo` (String)
     - `autor` (String)
     - `anoPublicacao` (Int)
     - `disponibilidade` (Bool) - Onde `true` significa que o livro está disponível e `false` significa que está emprestado.
2. **Cadastro de Livros (Array)**:
   - Crie uma lista (um **array**) para armazenar vários livros.
   - Crie uma função chamada `adicionarLivro` que receba os parâmetros de título, autor, ano de publicação e disponibilidade. Ela deve adicionar um novo livro à lista.

3. **Visualização de Livros (Funções e Loops)**:
    - Crie uma função chamada `listarLivros` que exibe todos os livros cadastrados, mostrando o título, autor, ano de publicação e se o livro está disponível ou não.
    - Utilize um **loop** para iterar sobre o array de livros e mostrar as informações.
4. **Alteração de Disponibilidade (Switch Case)**:
    - Crie uma função chamada `alterarDisponibilidade` que permita ao usuário alterar o status de disponibilidade de um livro específico. O usuário deve passar o título do livro e o sistema deve mudar o status de disponibilidade (se está disponível para emprestado e vice-versa).
    - Use **switch case** para mostrar a alteração de status de forma clara, por exemplo, "O livro [título] foi emprestado!" ou "O livro [título] está disponível novamente."
5. **Verificação de Livro (Optionals)**:
    - Ao buscar um livro pelo título, use **optionals** para verificar se o livro foi encontrado na lista. Caso contrário, o sistema deve exibir uma mensagem dizendo que o livro não existe.

Exemplo de Fluxo:

1. O usuário chama a função `adicionarLivro` para adicionar alguns livros à lista.
2. Ele chama a função `listarLivros` para visualizar os livros cadastrados.
3. Ele então chama `alterarDisponibilidade` para mudar o status de disponibilidade de um livro.
4. Por fim, o usuário tenta buscar um livro pelo título, e o sistema retorna se o livro foi encontrado ou não.

Dicas:

- Use **optionals** para lidar com livros que não existem.
- Use **tuplas** para representar os livros ou dados de entrada (por exemplo, para passar os parâmetros de um livro para a função).
- Organize o código de maneira modular, com funções separadas para cada ação (adicionar, listar, alterar, buscar).

Exemplo de Saída Esperada:

```
Lista de Livros:
1. Título: O Senhor dos Anéis | Autor: J.R.R. Tolkien | Ano: 1954 |
Disponível: Sim
2. Título: Harry Potter | Autor: J.K. Rowling | Ano: 1997 |
Disponível: Não

Alterando disponibilidade do livro 'Harry Potter'...
```

O livro 'Harry Potter' foi emprestado!

Lista de Livros:
1. Título: O Senhor dos Anéis | Autor: J.R.R. Tolkien | Ano: 1954 | Disponível: Sim
2. Título: Harry Potter | Autor: J.K. Rowling | Ano: 1997 | Disponível: Não

Procurando pelo livro 'O Hobbit'...
Livro não encontrado.